

REPUBLIC OF CAMEROON

Peace – Work – Fatherland

MINISTRY OF SECONDARY EDUCATION

INSPECTORATE GENERAL OF EDUCATION

Inspectorate of Pedagogy in charge of the
Teaching of Computer Science



REPUBLIQUE DU CAMEROUN

Paix – Travail – Patrie

MINISTRE DES ENSEIGNEMENTS SECONDAIRES

INSPECTION GENERALE DES ENSEIGNEMENTS

Inspection de Pédagogie chargée de
l'enseignement de l'informatique

NATIONAL HARMONISED PROGRESSION SHEET FOR COMPUTER SCIENCE UPPER SIXTH

SCHOOL YEAR:

SCHOOL:

WEEKLY WORKLOAD: 8 periods

TEACHER:

GRADE:

TELEPHONE:

Term	Week	Module	Category of action	Lesson no	Lesson title	Objectives	Resources	Th	Pra	Dig
First Term	1	Computer Applications, Artificial Intelligence & Emerging Technologies And Their Socio-Economic Implications	Exploring ai concepts and emerging technologies	1	Introduction to Artificial Intelligence (AI) - AI Ethics and Responsible Use	• Define AI and explain its history and evolution. • Identify and describe the main types of AI. • Explain common applications of AI. • Explain ethical issues associated with AI. • Examine the potential risks and benefits of AI. • Discuss principles for the responsible use of AI.	AI simulations and AI tools			
				2	AI Techniques and Intelligent Systems	• Describe common AI techniques and their applications. • Explain the characteristics of intelligent systems. • Compare AI techniques and intelligent systems based on their applications				
				3	Machine Learning	• Define Machine Learning. • Explain the basic principles of Machine Learning. • Distinguish between supervised, unsupervised and reinforcement learning.				
				4	Developing AI Systems	• Explain the stages involved in developing an AI system. • Identify programming languages and tools used in AI development. • Explain why Python is widely used in AI development.				
	2			Selecting and manipulating	5	Integration Activity				
			6		Abstract Data Types	• Explain the concept and characteristics of ADTs. • Differentiate between ADTs and data structures. • Relate common data structures to the implementation of ADTs				

			abstract data types (adts) and data structures			• Explain the concept, characteristics and, operations of stack and queue ADTs. • Compare stack and queue ADTs based on their principles of operation and typical applications. • Identify how abstract data types support data structures used in AI applications.				
First Term		DATA STRUCTURES AND ALGORITHM DESIGN 2		7	Binary Tree Concepts and Representation	• Explain the concept, characteristics, and terminology of binary trees. • Represent binary trees using suitable diagrammatic and storage methods. • Perform preorder, inorder, and post order traversals on binary trees. • Explain the characteristics and organisation of Binary Search Trees (BSTs). • Perform searching, insertion, and deletion operations on BSTs. • Explain how tree structures can represent decisions in simple AI systems.				
				8	Applications of ADTs	• Explain the role of stacks in expression processing. • Evaluate postfix expressions (RPN) using stack operations. • Convert infix expressions to postfix and prefix expressions using stack operations. • Construct expression trees from arithmetic expressions. • Convert arithmetic expressions into prefix, infix, and postfix notations using preorder, inorder, and postorder traversals of expression trees. • Identify how stacks and queues can support data processing in AI applications.				
				9	Integration Activity					
	3	Applying searching and sorting techniques.	Applying searching and sorting techniques.	10	Searching Algorithms	• Explain the purpose and principles of searching algorithms. • Trace linear search and binary search algorithms. • Compare searching algorithms based on their approach and efficiency. • Explain how search techniques are used to explore solutions in AI problems.				
				11	Sorting Algorithms	• Explain the purpose and principles of sorting algorithms. • Trace bubble sort, selection sort, and insertion sort. • Compare the three algorithms based on efficiency and suitability. • Identify how sorting supports efficient preparation of data for AI tasks.				
		REPRESENTING DATA IN A COMPUTER	Representing data in a computer	12	Representation of Fractional Numbers 1	• Explain fixed-point and floating-point representations of fractional numbers. • Describe the components and characteristics of fixed-point and floating-point representations. • Represent fractional decimal numbers in fixed-point and floating-point binary representations. • Relate numerical representation and precision to data used by AI systems.				
	4			13	Representation of Fractional Numbers 2	• Convert floating-point binary representations into decimal values.				

First Term						• Normalise floating-point representations using appropriate formats. • Explain the causes and effects of precision errors in floating-point representation. • Explain why numerical precision matters when AI systems process numerical data.				
				14	Integration Activity					
				15	Boolean Algebra Principles	• Explain the role of Boolean algebra in representing and analysing digital circuits. • Derive Boolean expressions from logic circuits and truth tables. • Apply Boolean algebra laws and theorems to simplify Boolean expressions. • Relate Boolean logic to simple rule-based AI decision systems.				
	5	ANALYSING DIGITAL LOGIC SYSTEMS	Analysing digital logic systems	16	Standard Forms and Logic Minimisation	• Explain standard forms of Boolean expressions, including Sum-of-Products (SOP) and Product-of-Sums (POS). • Derive SOP and POS expressions from truth tables and logic circuits. • Use Karnaugh maps to simplify Boolean expressions and minimise logic circuits. • Explain how logic minimisation can support efficient rule-based AI systems.				
				17	Integration Activity					
		PROGRAMMING 2	Write, debug and run programs in C	18	Setting up a C Programming Environment	• Describe how to install the IDE • Install a compiler • Describe the interface of the name IDE • Explain C, compile-run cycle, components of a C program • Write and run “Hello World”; fix basic compile errors • Identify how programming tools can be used to develop simple AI-related programs.				
				19	Implementing basic instructions	• Declare variables, choose correct data type, use constants • Format output correctly; read input safely • Evaluate arithmetic/relational/logical expressions • Use basic programming instructions to prepare simple data-processing tasks for AI applications.				
				20	Implementing Selection	• Write decision programs (pass/fail, max of two) • Handle multi-condition logic correctly • Manipulate nested selection structure • Trace values of variables during selection with a for loop • Implement decision rules that illustrate simple AI decision-making.				
				21	Implementing Iteration	• Differentiate between pre-test, post-test loop and counting loop • Choose a more efficient looping method to solve a problem • Trace values of variables during iteration with a while/do-while loop and for loop • Use iteration to process data in a simple AI-related task.				
	6			22	Strings & character handling	• Identify string libraries (string.h, ctype) • Use some string functions like strcmp, strcpy, strlen, strcat,.. • Manipulate strings (declare, initialize, input/output...)				

First Term	7	PROGRAMMING 2				• Manipulate text data for a simple AI-related application.				
					Evaluation					
					Remediation					
				23	Implementing 1D Arrays with loops	• Declare & initialize 1D arrays with different data types and structures. • Manipulate array elements using appropriate loops (for loop) • Compute sum/average/min/max using loops • Use arrays to store and process data that could be used by an AI system.				
				24	Implementing 2D Arrays with loops	• Declare & initialize 2D arrays of primitive data types • Manipulate array elements using appropriate nested loops (for loop) • Compute matrix addition and multiplication • Use two-dimensional arrays to represent data for a simple AI-related task.				
				25	Implementing Records and list	• create a struct to manipulate data • Manipulate array of records • Declare a list in a programming language • Organise structured data that could be used as input to an AI application.				
	8	SOFTWARE DEVELOPMENT	Evaluating software development approaches.	26	Implementing Subroutines	• Write and call subroutines(functions/procedures) • Build programs using subroutines (e.g., sum, max) • Use appropriate parameter passing to declare and call functions (pass by value or by reference) • Use recursive functions to solve problems • Design reusable functions that support simple AI-related data processing.				
				27	Integration Activity					
				28	Software Development Activities and Models	• Explain the purpose of software development models. • Select an appropriate development model for a given situation and justify the choice. • Describe the activities of software development and outline the importance of developing software. • Explain the advantages and limitations of each software-development model and assess criteria for applicability of different models. • Identify where AI tools can support selected activities in the software development life cycle.				
				29	Requirements Analysis and Specification	• Explain software requirement analysis and illustrate requirements of specific software. • Outline elicitation, validation, specification and verification activities in requirements analysis. • Write a requirements specification document and compare technical and user requirements. • Identify basic requirements for a software system that uses an AI component.				
First Term		SOFTWARE DEVELOPMENT								

				30	Software Design - Software Design Stages and Documentation	• Explain the importance of software design in software development. • Apply modular, data, process, and interface design principles to software solutions. • Analyse cohesion, coupling, software reuse, and APIs in software design. • Identify software components to be designed; explain and specify design elements. • Identify interface, architectural and detailed design stages and produce a design document. • Include an appropriate AI component when designing a software solution for a given problem.				
	9			31	Software Development Tools	• Explain the purpose of tools used in software development. • Differentiate between CASE tools, IDEs, and version control systems. • Select suitable development tools for a given software development situation. • Explain the concept and importance of software quality. • Identify key characteristics of quality software. • Identify AI-assisted development tools and explain their appropriate use.				
				32	Software Quality, Acquisition and Evaluation	• Evaluate software quality based on specified quality characteristics. • Evaluate the demands involved in acquiring software, including financial, legal and security considerations. • Source/search for software in the community or online. • Evaluate software for a specific task in terms of suitability and limitations. • Consider accuracy and reliability when evaluating software that uses AI.				
				33	Software Verification, Validation and Testing	• Explain software verification and validation. • Explain testing mechanisms and describe methods such as code review and static code analysis. • Apply and differentiate software-testing methods. • Identify simple tests for checking the reliability of an AI-supported software feature.				
				34	Software Project Management and Reporting	• Explain project-management activities: proposal writing, planning, monitoring, personnel selection, and evaluation reporting. • Produce a report covering software-development stages, outsourcing and software-management activities.				
	10			35	Software Reuse, Teamwork and Maintenance	• Explain reasons for developing new software/systems and choosing a design procedure; explain software reuse; analyse and amend existing programs. • Explain team-work purpose in program development; collect user opinions; evaluate data from different sources and draw/reference conclusions. • Explain debugging techniques and differentiate corrective and preventive maintenance. • Document projects.				

First Term	11	SOFTWARE		36	Integration Activity					
			Categories of software	37	classification of software	- Identify attributes of a good software - Classify software (system and application) - Identify application software and categories - Evaluate demands in acquiring software - Evaluate software for a specific task (suitability and limitations)				
			Exploring system software concepts.	38	Introduction to System Software	- Explain the meaning and purpose of system software. - Differentiate system software from application software. - Classify system software into major categories.				
				39	System and Utility Software	- Describe common system and utility programs: device drivers, file management/transfer, linking loaders, and maintenance utilities. - Choose suitable system software for a given purpose. - Identify the functions of system and utility programs such as linking loaders, transfer programs and subroutine libraries. - Identify system resources needed to run common AI applications.				
				40	Types and Evolution of Operating Systems	- Describe different types of operating systems and their characteristics. - Explain the evolution of operating systems from early batch-processing systems to modern operating systems. - Relate different types of operating systems to their applications and computing requirements - Explain how operating-system capabilities support modern AI applications.				
				41	Operating System Interfaces	- Explain the purpose of an operating system interface. - Describe and compare different types of operating system interfaces, including their characteristics, advantages, and limitations. - Select an appropriate operating system interface for a given user or computing situation. - Identify how interfaces can support interaction with AI-enabled applications.				
				42	Operating System Installation and File Storage	- Explain how operating systems are installed. - Describe file-storage methods. - Explain how storage resources support datasets used by AI applications.				
	43		Integration Activity							
	12		Managing processes.	44	Process Management Basics	- Explain the concept of a process and distinguish between a process and a program. - Describe and illustrate the states of a process and the transitions between them. - Explain process creation, process termination, and the purpose of context switching. - Explain how the operating system uses stacks when switching context during function calls.				
					Evaluation					
					Remediation					

Second Term	13	SOFTWARE		45	Process Scheduling	• Explain the purpose of process scheduling and describe the roles of the scheduler and dispatcher in process scheduling. • Distinguish between pre-emptive and non-pre-emptive scheduling. • Explain the metrics used to evaluate scheduling algorithms.				
				46	Non-preemptive Scheduling	• Explain the principles of non-pre-emptive process scheduling. • Describe and apply common non-pre-emptive scheduling algorithms, including FCFS, SJF, and non-pre-emptive Priority Scheduling. • Construct Gantt charts and calculate scheduling performance metrics for non-pre-emptive scheduling algorithms. • Explain starvation and ageing of processes.				
				47	Pre-emptive Scheduling	• Explain the principles of pre-emptive process scheduling and distinguish it from non-pre-emptive scheduling. • Describe common pre-emptive algorithms: SRTF, pre-emptive Priority, and Round Robin. • Construct Gantt charts and calculate scheduling performance metrics for pre-emptive scheduling algorithms. • Apply the Round Robin algorithm to a set of processes and construct Gantt charts using different time-quantum values and evaluate the effect of time quantum.				
				48	Process Synchronisation	• Explain the need for process synchronisation and the problems that can occur when processes access shared resources concurrently. • Explain the concepts of critical section, race condition, and mutual exclusion. • Describe and apply common process synchronisation mechanisms, including locks, semaphores, and monitors. • Differentiate communicating and non-communicating concurrent processes. • Identify the race condition in a given real-life scenario and explain methods to resolve it.				
Second Term	14	SOFTWARE		49	Deadlock Handling	• Explain the concept of deadlock and the conditions necessary for its occurrence. • Sketch a resource-allocation graph (RAG) for a given scenario and use it to determine whether deadlock is possible. • Describe and distinguish between deadlock prevention, avoidance, detection, and recovery strategies.				
				50	Integration Activity					
			Managing memory resources.	51	Memory Management Basics	• Explain the need to manage several processes in a system with physically limited main memory. • Differentiate between logical (virtual) addresses and physical addresses and their corresponding address spaces. • Describe process loading, address binding, and swapping.				
				52	Contiguous Memory Allocation	• Describe fixed and variable partitioning as contiguous memory allocation schemes. • Explain internal and external fragmentation and the limitations of contiguous memory allocation.				

Second Term	15					• Apply first-fit, best-fit, and worst-fit placement policies to allocate processes to memory partitions.				
				53	Non-contiguous Memory Allocation	• Explain the need for non-contiguous memory allocation and distinguish it from contiguous memory allocation. • Describe paging and segmentation as non-contiguous memory allocation schemes. • Compare the advantages and limitations of paging and segmentation.				
				54	Virtual Memory and Demand Paging	• Explain the concept and purpose of virtual memory and distinguish virtual memory from physical memory. • Describe how demand paging loads pages into main memory when they are required. • Explain page faults and describe how the operating system handles a page fault.				
				55	Integration Activity					
				56	I/O Device Management	• Explain the role of the operating system in managing input and output devices. • Describe the functions of device drivers, buffering, and spooling in I/O management. • Explain how the operating system coordinates communication between applications, I/O devices, and the processor.				
	16		Managing input/output operations and file systems	57	I/O Transfer Mechanisms	• Describe programmed I/O (polling), interrupt-driven I/O, and direct memory access (DMA). • Explain how the operating system handles hardware interrupts during I/O operations. • Compare programmed I/O, interrupt-driven I/O, and DMA in terms of processor involvement and efficiency.				
				58	File Systems & Management	• Explain the role of the operating system in managing files and directories. • Describe how files are named, organised, and stored in directories. • Describe common file and directory attributes and operating system operations performed on files. • Explain the purpose of a file system and how it organises and manages files on storage devices. • Compare FAT, NTFS, and ext file systems in terms of organisation, features, and typical use.				
				59	Mass Storage and File Access Methods	• Explain characteristics of mass-storage devices. • Determine ratings of file-access methods. • Create and manage files in the computer. • Differentiate sequential and direct access.				
				60	OS Security and Error Management	• Describe the security strategy in an operating system. • Outline types and sources of errors in an operating system • Describe the error-management strategy.				
				61	Integration Activity					
	17	INFORMATION SYSTEMS	Supporting decision-making in organisations	62	Introduction to Information System	• Define an information system and explain the need for information systems in organisations. • Describe the main components of an information system. • Explain the characteristics of good information and its importance to organisations.				

Second Term	18					- Define a community problem for which an information system can be developed and discuss the need for an IS in an organisation. - Illustrate management levels and information flow within an organisation.				
				63	Data Capture	- Define data capture and distinguish it from data processing. - Describe common methods of data capture and their applications				
				64	Data Verification and Validation	- Define data verification and data validation. - Describe common methods of data verification and validation. - Apply appropriate verification and validation techniques to ensure the accuracy and integrity of captured data.				
				65	Organisational Levels and Information Systems	- Identify the main levels of an organisation and the people involved at each level. - Describe the types of information generated and required at each organisational level. - Describe the information systems used at each organisational level.				
				66	Integration Activity					
					Evaluation					
					Remediation					
	19	INFORMATION SYSTEMS	Developing information systems	67	System Development Process	- Identify the stages of the System Development Life Cycle (SDLC). - Describe the purpose and activities of each stage. - Identify the major deliverables produced at each stage - Explain how an existing system can be transformed into a computerised system. - Describe phases required to develop an IS and implement the steps to develop one for an organisation in the community.				
				68	System Design Tools	- Explain the purpose of modelling tools in system development. - Describe and interpret UML diagrams and data-flow diagrams. - Use pseudocode or structured English to represent system processes. - Draw a data-flow diagram and set up a library or school management information system as a project.				
				69	System Testing	- Explain the purpose of testing in system development. - Describe different types of test data and system testing. - Select appropriate test data to test system functionality.				
				70	Information System Management	- Identify measures for managing and protecting information systems. - Describe procedures for maintaining, backing up and securing information systems. - Explain the importance of documentation, user support and system maintenance. - Explore a GIS environment; identify GIS interface elements; browse/close/save maps; query maps using Identify/Find; and organise map layers.				
	20	DATABASE MODELING AND DESIGN		71	Database Fundamentals and Data Organisation	- Explain the concept and purpose of a database. - Explain the role and functions of a DBMS, with examples. - Identify what qualifies an arrangement of data as a database; - outline advantages/disadvantages of computerised and non-computerised databases.				

Second Term				72	Database Models	• Explain the concept of a database model and its purpose in organising data. • Describe hierarchical, network, relational, and object-oriented database models. • Compare the characteristics, advantages, and limitations of different database models. • Sort out advantages of multiple-file database systems over flat-file systems.				
				73	Relational Database Models	• Describe the structure and characteristics of a relational database. • Explain the problems of flat-file systems and how the relational model addresses these problems. • Explain how primary, candidate, and foreign keys are used to identify and relate data. • Organise data into fields and records and check data entered into the system.				
				74	Database Architecture	• Explain the concept and purpose of database architecture. • Describe the three levels of database architecture: external, conceptual, and internal. • Explain how the three levels provide data abstraction and data independence.				
	21			75	Entity-Relationship Modelling	• Explain the purpose of E-R modelling and describe the components of an E-R diagram. • Explain the degree and cardinality of relationships in an E-R model. • Construct and interpret E-R diagrams for given database scenarios.				
				76	Relational Database Design and Relationships	• Explain the significance of relational database concepts; organise data into related tables; and select appropriate data types for fields. • Set up joins between tables and enforce constraints using RDBMS/SQL features. • Work with modification anomalies.				
				77	Database Normalisation and Functional Dependencies	• Explain the purpose of database normalisation and the problems caused by data redundancy. • Explain insertion, update, and deletion anomalies in relational databases. • Establish reasons for normalization and transform tables through 1NF, 2NF, 3NF and BCNF. • Identify functional dependencies and their significance. • Compare simple database designs with peers.				
				78	Database Design Stages and Data Modelling	• Identify the four database-design stages: conceptual, logical, normalization and physical design. • Determine attribute domains and candidate keys; decompose many-to-many relationships; establish a data dictionary. • Represent a collection of data using an Entity-Relationship model.				
	22			79	Database Integrity	• Explain the concept and importance of data integrity in databases. • Describe database integrity constraints, including entity integrity, referential integrity, and domain constraints (check constraints). • Describe the ACID and BASE approaches to database transactions.				
				80	Integration Activity					

Second Term	23	DATABASE MODELING AND DESIGN	Managing and retrieving information	81	SQL - Data Definition Language	• Explain the purpose of Data Definition Language (DDL) and describe common DDL statements. • Use CREATE, ALTER, and DROP statements to create and modify database structures. • Define tables using appropriate data types, keys, and integrity constraints.				
				82	SQL – Data Manipulation Language	• Explain the purpose of Data Manipulation Language (DML) and describe its main commands. • Use INSERT, UPDATE, and DELETE statements to modify data in database tables. • Apply appropriate conditions when modifying or deleting data from tables.				
				83	SQL - Data Query Language	• Explain the purpose and syntax of the SELECT statement for retrieving data from database tables. • Use WHERE and logical/comparison operators to filter retrieved data. • Use ORDER BY to sort retrieved data in ascending or descending order.				
				84	SQL – Joins	• Explain the purpose of SQL joins and distinguish between different types of joins. • Construct SQL statements using joins to retrieve related data from multiple tables.				
				85	Aggregate Functions	• Explain the purpose and operation of SQL aggregate functions. • Use COUNT(), SUM(), AVG(), MIN(), and MAX() to calculate summary values from database data. • Use GROUP BY and HAVING with aggregate functions to produce grouped and filtered summary results.				
				86	RDBMS Packages and Database Implementation	• Use RDBMS packages such as MS Access and Open/LibreOffice Base. • Create tables, forms and reports in an RDBMS; • import/export databases; • populate/retrieve data using queries and SQL; • implement joins and subqueries.				
	24			87	Web and Object-Oriented Database Integration	• Implement PHP/HTML to publish and connect a database-driven website. • Use an object-oriented database to store/retrieve information, link it to a website and enter data through the website.				
				88	Integration Activity					
Third Term	25	PRACTICAL PROJECT AND REVISION	Computer architecture & hardware organization	89	Computer Architecture & Hardware Organization	• Analyze the Von Neumann architecture, CPU components (ALU, CU, Registers), and system buses (Data, Address, Control). • Trace the Fetch-Decode-Execute instruction cycle and evaluate the memory hierarchy (Cache, RAM, Registers).				
				90	Instruction Set Architecture & Low-Level Processing	• Convert between data representations (Binary, Hexadecimal, Two's Complement, IEEE 754 Floating Point). • Analyze assembly language instructions, addressing modes, and CPU register operations.				

				91	Hands-on C Project: CPU Instruction Cycle & Memory Simulation	• Simulate CPU register changes and instruction execution steps using a logic/CPU simulator. • Trace memory access times and cache hit/miss rates within a 2-hour lab session.						
				92	Hands-on C Project: Assembly Code & Register Manipulation Lab	• Write and execute basic low-level routines (arithmetic, loops, conditional jumps) in an assembly simulator. • Debug register states and memory contents under a 2-hour time constraint using pre-built starter templates.						
	26		Programming, data structures & operating systems	93	Data Structures & C Programming Foundations	• Trace linear and binary search algorithms, and sorting techniques (Bubble, Selection, Insertion). • Declare and manipulate abstract data types (Stacks, Queues, Linked Lists, Binary Trees) using dynamic memory allocation in C.						
				94	Operating Systems & System Software	• Solve CPU scheduling problems (FCFS, SJF, Round Robin) and draw Gantt charts to calculate turnaround and waiting times. • Evaluate memory allocation techniques (First-fit, Best-fit, Paging) and analyze memory fragmentation issues.						
				95	Hands-on C Project: Data Indexing & Search Module	• Implement a C program that stores records in a Linked List or Binary Search Tree using pre-provided starter code. • Execute and test functions for dynamic record insertion, searching, and sorting within a 2-hour lab session.						
				96	Hands-on OS Project: CPU & Memory Allocation Simulator	• Build a spreadsheet model (or complete a programmatic shell) simulating FCFS/Round Robin CPU scheduling. • Calculate average turnaround time and test First-fit vs Best-fit placement for process memory requests.						
	Third Term			27	PRACTICAL PROJECT AND REVISION	Database systems & information retrieval	97	Relational Database Modeling & Normalization	• Construct Entity-Relationship Diagrams (ERD) from enterprise business scenarios. • Apply functional dependencies to normalize relational tables from 1NF up to 3NF/BCNF.			
			98				Database Querying & Information Retrieval with SQL	• Write DDL statements (CREATE, ALTER) and DML queries (INSERT, UPDATE, DELETE). • Formulate complex DQL queries using inner/outer JOINS, GROUP BY, HAVING, and aggregate functions.				
			99				Hands-on Database Project: ER Diagramming & Normalization Case Study	• Analyze a real-world case study (e.g., Hospital or School Management) and draw the normalized ERD using software. • Produce the relational schema and data dictionary table within the allotted 2-hour period.				
			100				Hands-on Database Project: SQL Database Creation & Web/Client Form Binding	• Execute a prepared SQL DDL script to generate tables and constraints in a DBMS. • Build core SQL queries and test data insertion through a simple HTML/PHP form interface template.				
28		Networking, security, software engineering & synthesis	101	Data Communication, Computer Networks & Security		• Distinguish OSI/TCP-IP layers, IP addressing (IPv4 subnets, CIDR notation), and routing fundamentals. • Analyze network security mechanisms (firewalls, encryption, attack vectors, and prevention strategies).						
			102	Software Engineering, SDLC & Theoretical Computer Science		• Compare SDLC models (Waterfall, Agile), testing levels, and project planning using PERT/Gantt charts (critical path). • Analyze Finite State Machines (FSM), state transition tables, and basic automata theory.						

				103	Hands-on Networking Lab: Subnetting & LAN Simulation	• Implement an IP subnetting scheme and configure IP addresses, subnet masks, and default gateways on a virtual network. • Test network connectivity using tools (ping, traceroute) within a 2-hour session.				
					Evaluation					
					Remediation					
Third Term	29		Revision							
	30									
	31									
	32									
	33									
	34									
	35									
	36									
							END OF PROGRAM			